

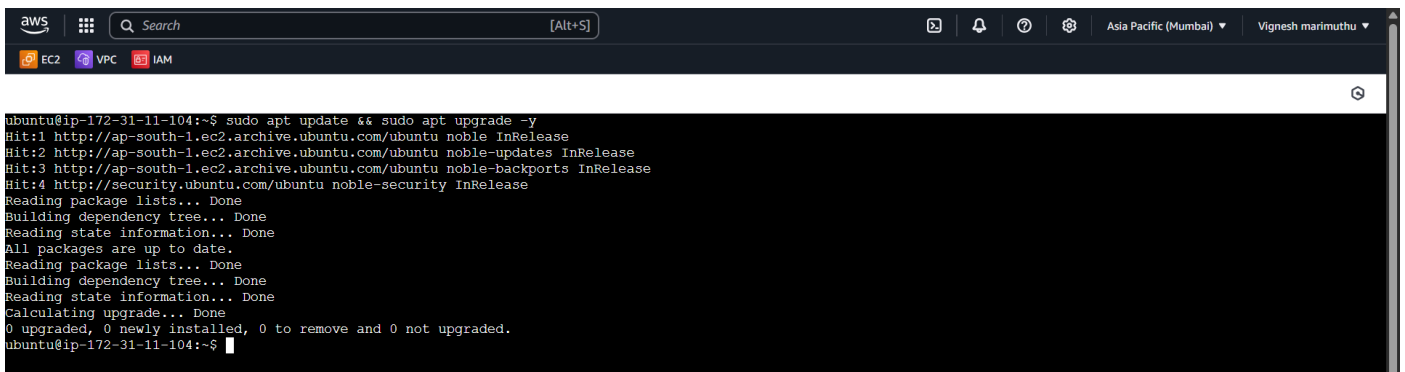
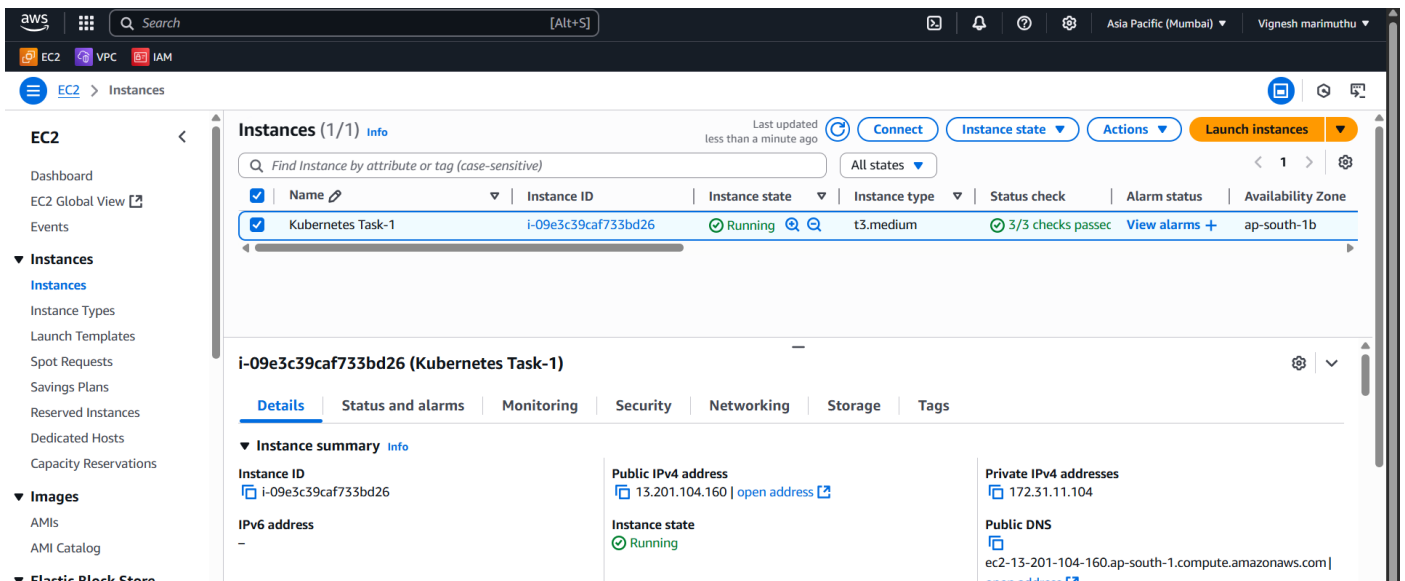
Kubernetes Task

Task Description:

Setup minikube at your local and explore creating namespaces (Go through official documentation).

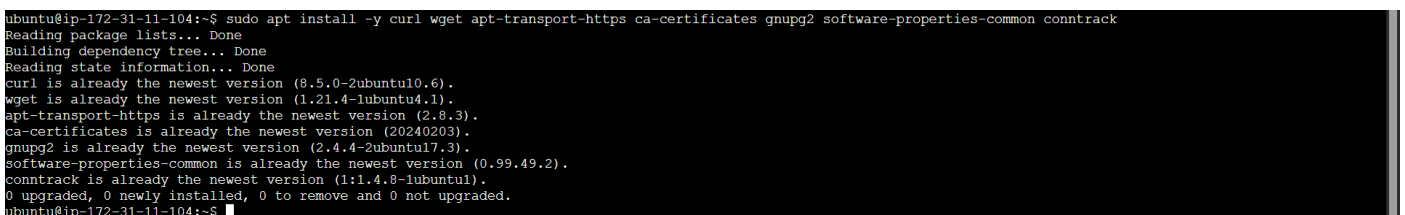
Solution:

✓ 1. Connected to EC2 Ubuntu instance & Update packages



- This updates Ubuntu system to the latest packages for security and performance.

✓ 2. Install dependencies (curl, apt-transport, etc.)



- These are basic tools needed for downloading and running **Minikube**, **Docker**, and **Kubernetes**.

✓ 3. Install Docker (needed for Minikube driver)

```
ubuntu@ip-172-31-11-104:~$ sudo apt install docker.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker.io is already the newest version (27.5.1-0ubuntu3~24.04.2).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu@ip-172-31-11-104:~$ docker --version
Docker version 27.5.1, build 27.5.1-0ubuntu3~24.04.2
ubuntu@ip-172-31-11-104:~$
```

i-09e3c39caf733bd26 (Kubernetes Task-1)

PublicIPs: 13.201.104.160 PrivateIPs: 172.31.11.104

- **Minikube** uses **Docker** to run a virtual **Kubernetes** cluster.
- We install **Docker** so **Minikube** can create containers.

```
ubuntu@ip-172-31-11-104:~$ docker --version
Docker version 27.5.1, build 27.5.1-0ubuntu3~24.04.2
ubuntu@ip-172-31-11-104:~$ sudo usermod -aG docker $USER
ubuntu@ip-172-31-11-104:~$ newgrp docker
ubuntu@ip-172-31-11-104:~$ curl -LO "https://dl.k8s.io/release/${curl -L -s https://dl.k8s.io/release/stable.txt}/bin/linux/amd64/kubectl"
```

- **Docker** is protected so random users can't use it.
- We need to **tell the system** that our user (ubuntu) is **allowed** to use Docker.
- Adding the user to the docker group gives the needed permission

✓ 4. Install kubectl (Kubernetes CLI)

```
ubuntu@ip-172-31-11-104:~$ curl -LO "https://dl.k8s.io/release/${curl -L -s https://dl.k8s.io/release/stable.txt}/bin/linux/amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
kubectl version --client
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 138 100 138 0 0 464 0 --:--:-- --:--:-- --:--:-- 464
100 57.3M 100 57.3M 0 0 109M 0 --:--:-- --:--:-- --:--:-- 109M
Client Version: v1.33.2
Kustomize Version: v5.6.0
ubuntu@ip-172-31-11-104:~$
```

i-09e3c39caf733bd26 (Kubernetes Task-1)

PublicIPs: 13.201.104.160 PrivateIPs: 172.31.11.104

- **kubectl** is the command-line tool to manage **Kubernetes** clusters.

✓ 5. Install Minikube

```
ubuntu@ip-172-31-11-104:~$ curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube version
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 126M 100 126M 0 0 12.9M 0 0:00:09 0:00:09 --:--:-- 16.7M
minikube version: v1.36.0
commit: f8f52f5de11fc6ad8244afac475e1d0f96841df1-dirty
ubuntu@ip-172-31-11-104:~$
```

i-09e3c39caf733bd26 (Kubernetes Task-1)

PublicIPs: 13.201.104.160 PrivateIPs: 172.31.11.104

- Download and install the latest version of **Minikube**.

✓ 6. Start Minikube with Docker driver

```
ubuntu@ip-172-31-11-104:~$ minikube start --driver=docker
* minikube v1.36.0 on Ubuntu 24.04
* Using the docker driver based on user configuration
* Using Docker driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.47 ...
* Downloading Kubernetes v1.33.1 preload ...
  > gcr.io/k8s-minikube/kicbase...: 502.26 MiB / 502.26 MiB 100.00% 44.03 M
  > preloaded-images-k8s-v18-v1...: 347.04 MiB / 347.04 MiB 100.00% 15.38 M
* Creating docker container (CPUs=2, Memory=2200MB) ...
* Preparing Kubernetes v1.33.1 on Docker 28.1.1 ...
  - Generating certificates and keys ...
  - Booting up control plane ...
  - Configuring RBAC rules ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
ubuntu@ip-172-31-11-104:~$
```

i-09e3c39caf733bd26 (Kubernetes Task-1)

PublicIPs: 13.201.104.160 PrivateIPs: 172.31.11.104

- This command will initialize a single-node Kubernetes cluster using Docker as the backend.

✓ 7. Check the cluster status

```
ubuntu@ip-172-31-11-104:~$ kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
minikube      Ready     control-plane  38s   v1.33.1
ubuntu@ip-172-31-11-104:~$
```

i-09e3c39caf733bd26 (Kubernetes Task-1)

PublicIPs: 13.201.104.160 PrivateIPs: 172.31.11.104

✓ 8. Create a Kubernetes namespace

```
ubuntu@ip-172-31-11-104:~$ kubectl create namespace dev-team
namespace/dev-team created
ubuntu@ip-172-31-11-104:~$
```

i-09e3c39caf733bd26 (Kubernetes Task-1)

PublicIPs: 13.201.104.160 PrivateIPs: 172.31.11.104

- **Namespace** helps separate resources between teams or projects in the same cluster.

✓ 9. List all namespaces

```
ubuntu@ip-172-31-11-104:~$ kubectl get namespaces
NAME          STATUS    AGE
default       Active    61s
dev-team      Active    10s
kube-node-lease Active    61s
kube-public   Active    61s
kube-system   Active    61s
ubuntu@ip-172-31-11-104:~$
```

i-09e3c39caf733bd26 (Kubernetes Task-1)

PublicIPs: 13.201.104.160 PrivateIPs: 172.31.11.104

✓ 10. Delete the namespace

```
ubuntu@ip-172-31-11-104:~$ kubectl delete namespace dev-team
namespace "dev-team" deleted
ubuntu@ip-172-31-11-104:~$
```

i-09e3c39caf733bd26 (Kubernetes Task-1)

PublicIPs: 13.201.104.160 PrivateIPs: 172.31.11.104

✓ 11. Stop Minikube

```
ubuntu@ip-172-31-11-104:~$ minikube stop
* Stopping node "minikube" ...
* Powering off "minikube" via SSH ...
* 1 node stopped.
ubuntu@ip-172-31-11-104:~$
```

i-09e3c39caf733bd26 (Kubernetes Task-1)

PublicIPs: 13.201.104.160 PrivateIPs: 172.31.11.104

- Namespace helps separate resources between teams or projects in the same cluster.

Steps I Performed

- I connected to my EC2 Ubuntu instance and **installed all dependencies** required to run **Kubernetes using Minikube**.
 - I installed **Docker as the Minikube** backend driver, along with **kubectl and Minikube binaries**.
 - Then, I started a **Minikube** single-node cluster using Docker.
 - Once **Minikube** was up and **running**, I used **kubectl** commands to create a new **namespace** called dev-team, listed it with other default namespaces, and verified the setup.
 - Finally, I optionally deleted the namespace and stopped the **Minikube cluster**.
-